Docker 复习：将 Cars CRUD 应用容器化

目标：将现有的 FastAPI + 纯 HTML + SQLite 应用打包成一个 Docker 镜像，作为一个容器运行，并将镜像发布到 Docker Hub。

开始之前：你应当已经了解镜像和容器是什么。这是一个简短、实用的后续练习，而不是 Docker 的完整介绍。请从现有 cars-crud/ 项目的根目录开始。

1. 快速回顾

术语	含义
Dockerfile	用于构建镜像的指令。
镜像	打包好的应用代码、Python 和依赖。
容器	镜像的一个运行实例。
仓库（Registry）	用于存储和分发镜像的在线服务。
Docker Hub	位于 https://hub.docker.com/ 的一个流行仓库。
标签	镜像版本标签，例如 cars-crud:v1。
卷（Volume）	删除容器后仍能保留的存储。

前提：安装 Docker Desktop（Windows/macOS）或 Docker Engine（Linux），并确保 Docker 服务正在运行。检查：

```
docker --version
docker info
```

2. 在中国更快地拉取 Docker 镜像（Azure 代理）

在中国访问 Docker Hub 有时会比较慢。本课程提供了一个 Azure 容器注册表代理，地址为 efreiwut.azurecr.io。它为拉取受支持的公共镜像提供了一条经由 Microsoft Azure 的替代路径。

例如，正常的 Docker Hub 拉取命令是：

```
docker pull python:3.12
```

在本地工作时，你可以改通过代理拉取同样的 Python 镜像和标签：

```
docker pull efreiwut.azurecr.io/dockerhub/library/python:3.12
```

标签仍然重要：:3.12 和 :3.12-slim 指的是不同的镜像。对于直接的 FROM 指令，
等价的选择有：

```
# 标准 Docker Hub 基础镜像
FROM python:3.12
```

```
# 通过课程 Azure 代理使用相同镜像标签
FROM efreiwut.azurecr.io/dockerhub/library/python:3.12
```

对于这个 Cars CRUD 应用，我们使用更小的 3.12-slim 变体，因此对应的代理路径是 efreiwut.azurecr.io/dockerhub/library/python:3.12-slim.

构建位置	推荐的镜像来源
你的电脑，且能正常访问 Docker Hub	python:3.12-slim
你的电脑，如果 Docker Hub 较慢	efreiwut.azurecr.io/dockerhub/library/python:3.12-slim
GitHub Actions 运行器	普通的 python:3.12-slim 基础镜像通常能正常工作；如果 Azure 代理可访问，也可以使用它。

重要区别：

- Azure 地址是受支持缓存镜像的拉取通道，而不是你发布 Cars CRUD 镜像所用的 Docker Hub 目的地。你仍然要将 YOUR_DOCKER_ID/cars-crud:v1 推送到你自己的 Docker Hub 仓库。
- 你并不因为本地使用了代理，就必须在 GitHub Actions 中也使用它。只要配置的注册表可达，两种路径都有效。
- 在生产环境中，团队通常使用自己的容器注册表进行拉取和推送，镜像名称类似 registry.example.com/team/cars-crud:v1。注册表主机名是镜像引用的一部分。
- 如果 Azure 代理返回授权错误或镜像不存在错误，请检查你的课程访问说明、/cache/library/ 路径和镜像标签。必须配置代理使其提供所请求的镜像。

下面的 Dockerfile 使用了一个构建参数，因此你无需每次编辑文件即可在 Docker Hub 和代理之间切换。

3. 创建 Dockerfile

在项目根目录、requirements.txt 旁边，创建一个恰好命名为 Dockerfile（无扩展名）的文件：

```
cars-crud/
├── Dockerfile           # 添加这个
├── .dockerignore        # 在下一节中添加
├── requirements.txt
├── app/
│   ├── main.py
│   ├── database.py
│   └── static/          # HTML、CSS、JavaScript
└── tests/
```

复制这个完整的 Dockerfile：

```
ARG BASE_IMAGE=python:3.12-slim
FROM ${BASE_IMAGE}

WORKDIR /app

COPY requirements.txt ./
RUN python -m pip install --no-cache-dir -r requirements.txt

COPY app/ ./app/

RUN mkdir -p /data && chown 10001:10001 /data
USER 10001:10001
ENV CARS_DB_PATH=/data/cars.db

EXPOSE 8000

CMD ["python", "-m", "uvicorn", "app.main:app", "--host", "0.0.0.0", "--
port", "8000"]
```

每条 Dockerfile 指令的作用

指令	在我们的应用中
ARG	声明构建时使用的基础镜像引用（默认：Docker Hub）。
FROM	从所选的 Python 3.12 slim 基础镜像开始。
WORKDIR	将 /app 设为镜像内部的工作目录。
COPY	先复制 Python 依赖，然后复制 app/ 代码和 HTML 文件。
RUN	在构建镜像期间执行命令（安装包、创建 /data）。
USER	以无特权的数字用户（而非 root）运行应用。
ENV	将 SQLite 路径设为容器内的 /data/cars.db。
EXPOSE	声明应用监听 8000 端口；它并不会发布该端口。

指令	在我们的应用中
CMD	指定容器启动时执行的命令。

注意两个细节：

- 服务器在容器内绑定到 0.0.0.0，这样发布的端口才能访问到它。这不同于在容器内使用 127.0.0.1。
- 我们先复制 requirements.txt，再复制源代码，这样当只有应用代码变化时，Docker 可以复用依赖安装层。我们只安装应用依赖，而不安装测试工具。

其他需要认识的有用指令：ENTRYPOINT（主可执行文件）和 HEALTHCHECK（容器健康检查）。本项目不需要它们。

4. 创建 .dockerignore

在 Dockerfile 旁边创建 .dockerignore：

```
.git/
.venv/
__pycache__/
*.py[cod]
.pytest_cache/
tests/
*.db
*.db-*
*.sqlite*
.env
```

这会将 Git 历史、本地 Python 环境、测试文件和本地 SQLite 数据库排除在 Docker 构建上下文之外。切勿把你的本地 cars.db、密码或访问令牌打进镜像。

5. 构建并运行容器

从项目根目录运行（最后的 . 表示“使用此目录作为构建上下文”）：

```
docker build -t cars-crud:local .
```

如果你在中国的本地电脑上访问 Docker Hub 较慢，可以仅为这次构建使用课程的 Azure 代理：

```
docker build --build-arg
BASE_IMAGE=efreiwut.azurecr.io/dockerhub/library/python:3.12-slim -t cars-
crud:local .
```

构建参数会为本次调用替换 Dockerfile 默认的 BASE_IMAGE。GitHub Actions 在构建已提交的 Dockerfile 时可以保持默认的 python:3.12-slim；无需编辑或添加额外的构建参数。生成的应用以相同方式打包。

为 SQLite 使用一个持久的命名卷来启动一个容器：

```
docker run -d --name cars-app -p 127.0.0.1:8000:8000 --mount
type=volume,source=cars-data,target=/data cars-crud:local
```

打开：

- 应用：http://localhost:8000/
- API 文档：http://localhost:8000/docs

使用 HTML 页面创建一辆车。它的 JavaScript 会调用 FastAPI 端点；两者都由同一个容器提供。无需单独的前端服务器或数据库容器。

理解 docker run 的各种标志：

标志	含义
`-d`	在后台运行。
`--name cars-app`	给容器起一个易于使用的名字。
`-p 127.0.0.1:8000:8000`	将本地 8000 端口连接到容器的 8000 端口，可从本机访问。
`--mount type=volume,source=cars-data,target=/data`	将 SQLite 数据库保存在一个命名的 Docker 卷中。
`cars-crud:local`	要启动的镜像。

Dockerfile 中的 CARS_DB_PATH 值指向 /data/cars.db。没有卷的话，删除容器也会删除其 SQLite 数据。卷与容器是分开保存的。

验证持久性：

```
docker stop cars-app
docker rm cars-app
docker run -d --name cars-app -p 127.0.0.1:8000:8000 --mount
type=volume,source=cars-data,target=/data cars-crud:local
```

刷新应用。之前创建的车辆应当仍然存在。

6. Docker CLI 速查表

命令	用途
`docker build -t cars-crud:local .`	根据本地 Dockerfile 构建镜像。
`docker image ls`	列出本地镜像。
`docker ps`	列出运行中的容器。
`docker ps -a`	列出运行中和已停止的容器。
`docker logs cars-app`	查看容器输出和错误。
`docker stop cars-app`	停止一个运行中的容器。
`docker start cars-app`	重新启动一个已停止的现有容器。
`docker rm cars-app`	删除已停止的容器，但不删除其命名卷。
`docker volume ls`	列出命名的 Docker 卷。
`docker exec -it cars-app sh`	在运行中的容器内部打开一个 shell。
`docker tag SOURCE TARGET`	给镜像起另一个名称/标签。
`docker push IMAGE`	将镜像上传到注册表。
`docker pull IMAGE`	从注册表下载镜像。

要实时跟踪应用日志，运行 docker logs -f cars-app，用 Ctrl+C 停止跟踪。这
不会停止运行中的容器。

7. 创建 Docker Hub 账户和仓库

1. 前往 https://hub.docker.com/ 并选择 Sign up。创建一个 Docker 账户并验证它。记录
   你的 Docker ID（用户名）。
2. 登录并打开 My Hub > Repositories > Create repository。
3. 选择你的 Docker ID 作为命名空间，并将仓库命名为 cars-crud。
4. 选择 Public 以获得易于分享的练习镜像，或者选择 Private 以限制访问。
   创建该仓库。
5. 在 Account settings > Personal access tokens 下，生成一个具有 Repo Read & Write
   权限和合适过期时间的令牌。显示时复制并妥善保存。你稍后
   在 GitHub Actions 练习中会再次用到它。

Docker Hub 仓库（YOUR_DOCKER_ID/cars-crud）存储容器镜像，而你的
GitHub 仓库存储源代码。

在终端中认证

```
docker login --username YOUR_DOCKER_ID
```

当提示输入密码时，粘贴 Docker Hub 的个人访问令牌，而不是你的 GitHub
密码。切勿将该令牌粘贴到 Dockerfile、Git 提交或截图中。

标记并推送镜像

在每个命令中用你的 Docker Hub 用户名替换 YOUR_DOCKER_ID：

```
docker tag cars-crud:local YOUR_DOCKER_ID/cars-crud:v1
docker push YOUR_DOCKER_ID/cars-crud:v1
```

名称遵循 NAMESPACE/REPOSITORY:TAG 格式。打开你的 Docker Hub 仓库，验证 v1 标签已出现。

可选：你也可以给同一镜像打上 latest 标签，但要记住 latest 只是一个标签名；它并不会自动找到最新版本。

8. 拉取你发布的镜像

在安装了 Docker 的电脑上（或删除本地标记的镜像后），你可以下载它：

```
docker pull YOUR_DOCKER_ID/cars-crud:v1
```

你可以使用第 5 节相同的 docker run 选项运行它，把 cars-crud:local 替换为 YOUR_DOCKER_ID/cars-crud:v1。如果你的仓库是私有的，拉取镜像的电脑必须具有访问它的权限。

9. 常见问题

问题	检查什么
无法连接到 Docker 守护进程	启动 Docker Desktop 或 Docker 服务。
无法读取 `Dockerfile`	从包含 Dockerfile 的目录运行 docker build。
找不到 Python 模块	检查 requirements.txt 和 app/ 是否存在于构建上下文中。
浏览器无法连接	检查 docker ps、docker logs cars-app 和 -p 映射。
8000 端口已被占用	停止本地 Uvicorn 进程或更改主机端口（例如，`127.0.0.1:8001:8000`).
权限被拒绝写入 SQLite	使用挂载在 /data 的命名卷；自定义 bind 挂载可能需要为用户 10001 调整所有权。
Docker Hub 推送被拒绝	检查命名空间、仓库名称、令牌权限和 docker `login`.

问题	检查什么
Azure 代理拉取失败	确认镜像名称/标签和 efreiwut.azurecr.io/dockerhub/library/ 路径；检查代理可用性或课程访问权限。

重要：这是一个无认证的练习应用。学习期间请将其端口绑定到 127.0.0.1；不要在没有添加访问控制的情况下将其公开暴露。

下一步：DOCKER_PUBLISHING_EXERCISE.md 会使用本指南中的 Dockerfile 和 Docker Hub 仓库。

参考

- Dockerfile 参考
- Docker Python 指南
- 创建 Docker Hub 账户
- 创建 Docker Hub 仓库
- Docker Hub 个人访问令牌